WILLIS PORT

Codex Implementation Brief - Phase 2 Revised

Localhost-First Development, Invoice Draft Workspace, Issue-and-Deliver, Containers, Payments, Receipts and Dashboard

This document supersedes the earlier Phase 2 brief. Phase 1 remains frozen and verified.

1. HARD FREEZE - PREVIOUS WORK MUST NOT BE CHANGED

Codex instruction - non-negotiable: The previously completed and verified work is frozen. We are moving forward. Do not redesign, rename, remove, replace, simplify or re-implement the existing shipment classification, ShippingRate logic, KG/CBM pricing authority, chargeable measurement handling, pricing approval protection, invoice calculations, migration history, invoice PDF calculation logic or Firebase/Neon deployment workflow.

- Make Phase 2 additive.
- Preserve all verified Phase 1 behavior unless an explicit Phase 2 task below requires the smallest possible extension.
- Do not refactor unrelated code while implementing these tasks.
- Do not change existing rates or pricing rules.
- Do not reintroduce Freight Charge into active pricing totals.
- Do not alter previously approved pricing records or recalculate historical issued invoices from mutable shipment values.
- Do not replace existing Prisma models when an extension/relation is sufficient.

1.1 LOCALHOST-FIRST DEVELOPMENT RULE

NEON FREEZE DURING ACTIVE DEVELOPMENT: All Phase 2 coding, Prisma schema changes, migrations, seeds, data edits, cleanup and functional testing must be performed only on localhost using the local PostgreSQL database and the normal .env file.

- Do not run Phase 2 migrations against .env.review / Neon while features are being developed.
- Do not seed, delete, reset, update or test transactional data directly in Neon during active development.
- Do not use Neon as the normal development database.
- Codex must use the local PostgreSQL database for all schema iteration and test data.
- Neon migration/deployment is a separate final release step.
- Do not touch Neon until the Phase 2 feature set is complete, local tests pass, the build passes, and explicit approval is given.

2. Verified Baseline - Preserve Exactly

- Next.js App Router + TypeScript + Prisma + PostgreSQL.
- ShippingRate is the database source of truth for structured pricing.
- Structured pricing key: shippingMode + serviceType + goodsCategory.
- Shipment stores current operational facts including weightKg, chargeableWeightKg, actualCbm and chargeableCbm.
- ShipmentPricing stores historical pricing snapshots.
- KG pricing uses Shipment.chargeableWeightKg server-side.
- CBM pricing uses Shipment.chargeableCbm server-side.
- Approved pricing cannot be silently replaced.
- Invoice PDF generation works and reflects approved pricing.
- BusinessSettings supplies Willis Port company/contact details.
- Firebase App Hosting deploys automatically from GitHub main.

- Neon review currently reflects the verified Phase 1 baseline and must remain untouched during Phase 2 development.

3. Phase 2 Direction

Turn the invoice draft page into the operational control point. Staff should be able to correct customer contact information, confirm shipment/container information, issue the final invoice through a chosen channel, automatically record the action, then proceed to payment and receipt. The dashboard must reflect this persisted workflow.

4. Invoice Draft Workspace - Pre-Issue Review

The invoice draft page is the final review point before an invoice becomes an issued business document. It must show customer, shipment, pricing and totals clearly while permitting limited factual corrections.

4.1 Customer Section - Add Edit Icon

- Add a small pencil/edit icon to the Customer section.
- Clicking the icon opens an inline editor or modal; staff should not leave the invoice workflow.
- Allow correction of customer name, phone, WhatsApp, email and address.
- Saving updates the real Customer database record, not temporary invoice-only text.
- Refresh the Customer section immediately after save.
- Email/WhatsApp/SMS action availability must re-evaluate immediately.
- A DRAFT invoice may reflect corrected customer data before issue.
- After issue, preserve the stored issued PDF as the historical record; future customer edits must not rewrite it.

4.2 Shipment Section - Limited Edit Capability

- Add a small shipment edit icon for factual shipment fields only.
- Allow correction of tracking number, description and container assignment.
- Reuse existing validated Shipment PATCH logic.
- Do not permit this action to modify approved pricing basis, approved rate, billable quantity or pricing totals.

5. Container Assignment - Required

Include a Container option in the Shipment area. Use the existing Container model and Shipment.containerId relation. Do not create a second container field or free-text substitute.

Requirement	Implementation	Rule
Shipment form	Container dropdown	Optional
Data source	Existing Container records	No arbitrary free text
Persistence	Shipment.containerId	Single source of truth
Invoice draft	Show assigned container	Read from Shipment relation
Invoice PDF	Show container identifier	Conditional: only when assigned
Reporting	Use relation for later container reports	Do not duplicate

For SEA/consolidated shipments staff can select an existing container. AIR shipments may remain unassigned. Omit the Container line from the final PDF when no container is assigned.

6. Issue-and-Deliver Architecture

The communication button itself becomes the business action. Staff should not generate a PDF, send it separately, then return later to update the database.

1. Invoice remains DRAFT while staff reviews it.
2. Staff chooses Email, WhatsApp, SMS, Mail or Print.
3. Server validates that the invoice is eligible to issue.
4. Server generates the final issued PDF.
5. The issued PDF must NOT show the DRAFT marking.
6. Store the final PDF permanently in cloud object storage.
7. Create/confirm an InvoiceDocument record for that exact PDF.
8. Create an InvoiceDelivery record for the selected channel.
9. Initiate or record the selected delivery action truthfully.
10. Update invoice status according to the successful action.
11. Dashboard updates from persisted records.

7. Final Invoice Document Storage

Do not store issued PDFs on the local server filesystem in production. During local development, storage integration may be mocked or configured locally, but the production target is Firebase Storage / Google Cloud Storage or the approved cloud object store.

Recommended Model: InvoiceDocument

Field	Suggested Type	Purpose
id	String @id @default(cuid())	Document identifier
invoiceId	String	Relation to Invoice
storagePath	String	Object-storage path
fileName	String	Issued PDF filename
generatedAt	DateTime @default(now())	Issue timestamp
version	Int @default(1)	Future version support
isIssued	Boolean @default(true)	Distinguish issued document

Recommended production storage path:

invoices/<customer-id>-<safe-customer-name>/<invoice-number>/<invoice-number>.pdf

- Generate the issued PDF once.
- Email, WhatsApp, SMS, Mail and Print should reference the same issued document.
- Do not regenerate a materially different PDF for every channel.
- Preserve the stored issued PDF even if Customer or Shipment later changes.

8. Invoice Delivery Tracking

Use delivery history rather than a single sent=true flag.

Field	Suggested Type	Purpose
id	String @id @default(cuid())	Delivery record
invoiceId	String	Relation to Invoice
invoiceDocumentId	String?	Exact issued PDF used
channel	InvoiceDeliveryChannel	EMAIL / WHATSAPP / SMS / MAIL / PRINT
recipient	String?	Email, phone or physical-mail label
status	InvoiceDeliveryStatus	PENDING / INITIATED / SENT / FAILED / PRINT_REQUESTED / MAIL_PREPARED
sentAt	DateTime?	Confirmed send time
failedAt	DateTime?	Failure time
failureReason	String?	Readable error
providerMessageId	String?	Future provider id
notes	String?	Staff note
createdAt	DateTime @default(now())	Audit timestamp

Delivery Truthfulness Rules

- Email is disabled when customer email is missing.
- WhatsApp is disabled when customer WhatsApp/phone is missing.
- SMS is disabled when customer phone is missing.
- Print is always available.
- Mail may represent physical mail/hand dispatch according to business process.
- Do not claim delivered merely because an icon was clicked.
- If a provider confirms send success, mark SENT.
- If no provider integration exists yet, record INITIATED / PRINT_REQUESTED / MAIL_PREPARED honestly.
- Each resend creates a new InvoiceDelivery record; never overwrite history.

9. Invoice Status Rules

- DRAFT while being reviewed.
- First successful/recorded issue action moves the invoice out of DRAFT.
- Use SENT and AWAITING_PAYMENT consistently with the existing lifecycle.
- Sending does not mean PAID.
- Payment completion moves invoice to PAID.
- Preserve CANCELLED behavior.
- All status transitions must be server-side and based on persisted records.

10. Payment and Receipt Workflow

- Add Record Payment action from invoice workspace.
- Validate payment amount and method server-side.
- Persist Payment before changing invoice status.
- Initial implementation must not falsely treat partial payment as fully paid.
- When fully paid, set Invoice.status = PAID.
- Generate receipt from persisted Invoice + Payment data.

- Receipt includes Willis Port details, receipt number, invoice number, customer, payment date, method, reference and amount.

11. Dashboard - Operational Reflection

Dashboard values must be derived from real records; do not create duplicated dashboard counters.

Dashboard Item	Persisted Rule / Source
New Requests	CustomerRequest.status = NEW
Shipments Received	Shipment.status = RECEIVED
Awaiting Pricing	No APPROVED ShipmentPricing
Pricing Approved	APPROVED ShipmentPricing
Draft Invoices	Invoice.status = DRAFT
Not Yet Issued	No issued InvoiceDocument / no delivery action
Sent / Awaiting Payment	Invoice.status = SENT or AWAITING_PAYMENT
Paid Invoices	Invoice.status = PAID
Email Actions	InvoiceDelivery.channel = EMAIL
WhatsApp Actions	InvoiceDelivery.channel = WHATSAPP
SMS Actions	InvoiceDelivery.channel = SMS
Print Actions	InvoiceDelivery.channel = PRINT
Total Invoiced GHS	Aggregate invoice total
Outstanding GHS	Unpaid invoice balance
Payments Received GHS	Aggregate Payment.amountGhs

Container Reporting Readiness

- Containers in transit / arrived when status data is available.
- Shipments per container.
- Customers represented in each container.
- Total CBM per container.
- Invoice value per container as a later reporting feature.

12. Direct Codex Implementation Instructions

A. Inspect Before Editing

- Inspect current invoice workspace, Customer display, Shipment display, invoice actions and PDF route.
- Inspect Customer PATCH/update route and reuse it.
- Inspect Shipment PATCH/update route and reuse it.
- Inspect Container model/routes/pages and Shipment.containerId relation.
- Inspect current BusinessSettings use in the PDF.

B. Customer Edit

- Add edit icon to Customer section on invoice draft page.
- Open inline/modal edit UI.
- Update real Customer record through validated server route.
- Refresh displayed data without leaving invoice workflow.
- Re-enable Email/WhatsApp/SMS actions immediately when contact details become available.

C. Container

- Add optional Container dropdown to appropriate Shipment create/edit area using existing Container records.
- Persist Shipment.containerId.
- Display assigned container on invoice draft.
- Include assigned container conditionally in invoice PDF.
- Do not add a free-text container field.

D. InvoiceDocument

- Add model/migration locally if not already present.
- During active development, test all schema/migration changes only against local PostgreSQL.
- Prepare production object-storage integration but do not touch Neon during development.
- Create one issued document record and reuse it across delivery channels.

E. Delivery Actions

- Add Email, WhatsApp, SMS, Mail and Print icons/buttons.
- Validate recipient/contact information server-side.
- Generate final non-DRAFT PDF at first issue action.
- Record InvoiceDelivery automatically.
- Do not require staff to return later to manually mark how the invoice was sent.

F. Payment / Receipt

- Implement Payment persistence and Record Payment action after issue/delivery is stable.
- Generate receipt from persisted records.

G. Dashboard

- Reflect invoice issue/delivery/payment state from local database during development.
- Add recent activity for invoice issued, delivery action, payment and receipt.
- Keep empty database state safe and useful.

13. Fresh Local Team-Test Reset Policy

If a fresh dataset is required during development, clear only transactional/client data in the LOCAL PostgreSQL database. Do not perform this reset against Neon during active development. Do not drop tables and do not delete configuration or migration history.

Clear Locally	Keep Locally
Customers	BusinessSettings
CustomerRequests	ShippingRate
Shipments	_prisma_migrations
ShipmentPricing	Schema / enums / indexes
Test Containers	Application configuration
Invoices / InvoiceLines	Rate-rule definitions
InvoiceDocument / InvoiceDelivery	
Payments / Receipts	

Safety rule: before destructive LOCAL cleanup, take a local logical backup if the test data should be retained. Neon remains frozen.

14. Local Quality Gates and Acceptance Tests

- Existing verified Phase 1 pricing tests still pass unchanged.
- Draft invoice shows Customer edit icon.
- Missing customer email can be added from invoice workspace and Email action becomes available.
- Shipment can be assigned to an existing Container.
- Assigned Container appears on invoice draft and final PDF.
- Unassigned Container is omitted from final PDF.
- First issue action generates one final PDF with no DRAFT marking.
- InvoiceDocument and InvoiceDelivery are persisted locally.
- Repeat delivery creates another history row without replacing the first.
- Payment and receipt workflow operates from persisted local data.
- Dashboard reflects the local workflow correctly.
- npx prisma format passes.
- npx prisma validate passes.
- npx prisma generate succeeds when needed.
- npx tsc --noEmit passes.
- npm run build passes.
- Full localhost end-to-end acceptance test is reviewed and explicitly approved.

15. FINAL RELEASE TO NEON - ONLY AFTER EXPLICIT LOCAL APPROVAL

This is a separate release phase. Codex must not perform these steps during normal Phase 2 development.

12. Confirm all Phase 2 work is complete and approved locally.
13. Confirm the working tree/code to be released is final.
14. Check pending Prisma migrations against .env.review without applying them.
15. Inspect every pending migration SQL manually.
16. Confirm no destructive or unintended migration is present.
17. Run prisma migrate deploy against Neon review only after approval.
18. Push the approved code to GitHub main.
19. Allow Firebase App Hosting to deploy from GitHub.
20. Perform one final hosted smoke/end-to-end test against Neon review.
21. Do not use Neon as a place to debug unfinished development.

16. Definition of Done

Phase 2 development is locally complete when Phase 1 remains unchanged and passing; customer corrections work from the invoice draft; container assignment flows into the invoice; issue-and-deliver produces one final non-DRAFT document and delivery history; payment and receipt are persisted; the dashboard reflects the complete operational state from the local database; TypeScript/build checks pass; and the full localhost workflow is explicitly approved. Only after that approval may the release be migrated to Neon review and deployed through GitHub/Firebase.

Final Codex reminder: Do not go backward, do not rebuild Phase 1, and do not touch Neon during active Phase 2 development.